KAVACH: Security-Governed Multi-Agent AI DevOps & Mission-Critical Banking Platform

Comprehensive Viva Preparation, Course Handout Code Navigation & Master Defense Dossier

> **Target Audience:** Viva Examiners (Dr. Soharab Hossain Shaikh & Mr. Pranshu Tiwari), Project-IV Lead (Prof. Anusha Chhabra), Technical Evaluators, and Viva Candidates

> **Repository Root Path:** `C:\Users\jaind\OneDrive\Desktop\uni\Ageentic&PRJ-IV`

> **Academic Context:** B.Tech CSE (AI & Data Science) · Academic Year 2026–27 · BML Munjal University, Gurugram

> **Courses Covered:**

> - **CSE3101 — Agentic AI** (3 Credits, LDP: 2-0-2) · Evaluators: Dr. Soharab Hossain Shaikh & Mr. Pranshu Tiwari

> - **Project-IV Capstone** (5 Credits) · Lead Coordinator: Prof. Anusha Chhabra

> **Core Author & Lead Architect:** Dhruv Jain (Enrollment No. 230532)

> **Contributors & Mid-Term Team:** Hemant Goyal, Rajat Yadav

> **Single Source of Truth Standard:** Measured reality is the source of truth. All documentation figures match code measurements directly verified by automated audit scripts.

Master Table of Contents

1. [Executive Summary & The Core Problem](#1-executive-summary--the-core-problem)

2. [Master Directory Structure & Clean Reorganized Anatomy](#2-master-directory-structure--clean-reorganized-anatomy)

3. [MASTER COURSE HANDOUT (CSE3101) TOPIC-BY-TOPIC IN-DEPTH PLAIN-ENGLISH GUIDE](#3-master-course-handout-cse3101-topic-by-topic-in-depth-plain-english-guide)

- [3.1 Prerequisite Review & Evolution to Autonomous Agents](#31-prerequisite-review--evolution-to-autonomous-agents)

- [3.2 Reasoning & Prompting Strategies (ReAct, CoT, ToT, GoT, LATS)](#32-reasoning--prompting-strategies-react-cot-tot-got-lats)

- [3.3 Agentic Design Patterns (Reflection, Planning, Tool Use, Swarms)](#33-agentic-design-patterns-reflection-planning-tool-use-swarms)

- [3.4 Agentic Retrieval-Augmented Generation (Code RAG & Vector Memory)](#34-agentic-retrieval-augmented-generation-code-rag--vector-memory)

- [3.5 CrewAI Framework Deep Dive (Agents, Tasks, Memory, Stateful Flows)](#35-crewai-framework-deep-dive-agents-tasks-memory-stateful-flows)

- [3.6 Google Agent Development Kit (ADK) Deep Dive (8-Hook Callbacks & Coordinator)](#36-google-agent-development-kit-adk-deep-dive-8-hook-callbacks--coordinator)

- [3.7 Emerging Protocols (Model Context Protocol / MCP & Agent-to-Agent / A2A)](#37-emerging-protocols-model-context-protocol--mcp--agent-to-agent--a2a)

- [3.8 Guardrails & Cyber Safety Master Deep Dive (All 9 Protective Subsystems)](#38-guardrails--cyber-safety-master-deep-dive-all-9-protective-subsystems)

- [3.9 Data Security & Privacy (Air-Gapped Local LLM vs Public Cloud Router)](#39-data-security--privacy-air-gapped-local-llm-vs-public-cloud-router)

- [3.10 Multimodal Agent Capabilities (Whisper Audio & Visual Prompt Defense)](#310-multimodal-agent-capabilities-whisper-audio--visual-prompt-defense)

- [3.11 Agent Evaluation & Benchmarking (Empirical Precision/Recall & Red-Teaming)](#311-agent-evaluation--benchmarking-empirical-precisionrecall--red-teaming)

- [3.12 Monitoring & Observability (SSE Telemetry & Tamper-Evident Merkle Trees)](#312-monitoring--observability-sse-telemetry--tamper-evident-merkle-trees)

- [3.13 Production Deployment (Docker, Kubernetes, Render, Batches)](#313-production-deployment-docker-kubernetes-render-batches)

- [3.14 Industry Domains & Use Cases (Bharat Core Banking SAGA ACID & DevOps)](#314-industry-domains--use-cases-bharat-core-banking-saga-acid--devops)

4. [UI WALKTHROUGH & FEATURE GUIDE: EARLIER MID-SEM TAB VS. NEWLY IMPROVISED TAB](#4-ui-walkthrough--feature-guide-earlier-mid-sem-tab-vs-newly-improvised-tab)

- [4.1 Tab 1: Earlier (Mid-Sem Foundation) Tab (v1.0 Baseline)](#41-tab-1-earlier-mid-sem-foundation-tab-v10-baseline)

- [4.2 Tab 2: Newly Added Improvised Innovations Tab (v3.0)](#42-tab-2-newly-added-improvised-innovations-tab-v30)

- [4.3 Side-by-Side Comparison & Audit Matrix (Earlier vs. Improvised)](#43-side-by-side-comparison--audit-matrix-earlier-vs-improvised)

5. [TWO-WAY NAVIGATION GUIDE (HOW TO DEMONSTRATE & SHOW THE CODE)](#5-two-way-navigation-guide-how-to-demonstrate--show-the-code)

- [5.1 Path A: Inside the Inbuilt Web IDE](#51-path-a-inside-the-inbuilt-web-ide)

- [5.2 Path B: Directly in VS Code (Shortcuts, File Paths & Highlighting)](#52-path-b-directly-in-vs-code-shortcuts-file-paths--highlighting)

6. [The 9 Defensive Stages & Subsystems (Detailed Engineering Architecture)](#6-the-9-defensive-stages--subsystems-detailed-engineering-architecture)

7. [Single Source of Truth (SSOT) Measured Metrics & Verification Reality](#7-single-source-of-truth-ssot-measured-metrics--verification-reality)

8. [Official University Evaluation Matrix (CSE3101: 30 Marks) & Student Submissions](#8-official-university-evaluation-matrix-cse3101-30-marks--student-submissions)

9. [Faculty Examiner Viva Defense Script (Dr. Shaikh & Mr. Tiwari)](#9-faculty-examiner-viva-defense-script-dr-shaikh--mr-tiwari)

10. [Operational Guide: How to Run, Test, and Verify Everything](#10-operational-guide-how-to-run-test-and-verify-everything)

11. [Deliberately Deferred Items, Scope Discipline & Phase 3 Roadmap](#11-deliberately-deferred-items-scope-discipline--phase-3-roadmap)

---

1. Executive Summary & The Core Problem

1.1 The Vulnerability of Naive Autonomous Coding Agents

Modern autonomous coding agents (such as Devin, SWE-agent, AutoPR, and Copilot Workspace) can decompose complex developer requests, inspect repository files, query vector databases, modify code across multiple directories, run continuous integration (CI) tests, and issue pull requests.

However, deploying unconstrained autonomous agents directly into production codebases and critical infrastructure creates catastrophic failure modes:

1. **Supply-Chain Poisoning & Slopsquatting:** Autonomous agents hallucinate non-existent external dependencies (e.g., `import fastapi_jwt_vault_security`) at an empirical rate of **19.7%** (Spracklen et al., USENIX Security 2025). Adversaries weaponize this behavior by registering these hallucinated package names on public package indices (PyPI, npm), achieving zero-click remote code execution when the agent executes its CI/CD pipeline.

2. **Credential & PII Exfiltration:** Agents inadvertently send unredacted corporate API keys, database credentials, AWS secrets, and Indian national identifiers (Aadhaar, PAN, phone numbers) to external, third-party model inference APIs (e.g., OpenAI, Anthropic), violating the statutory mandates of the **Digital Personal Data Protection Act, 2023 (DPDP Act)** and the **EU AI Act (2024/1689)**.

3. **Unbounded Blast Radius:** Without whole-repository Abstract Syntax Tree (AST) topological awareness, an agent modifying a peripheral helper function may silently break critical downstream services across module boundaries.

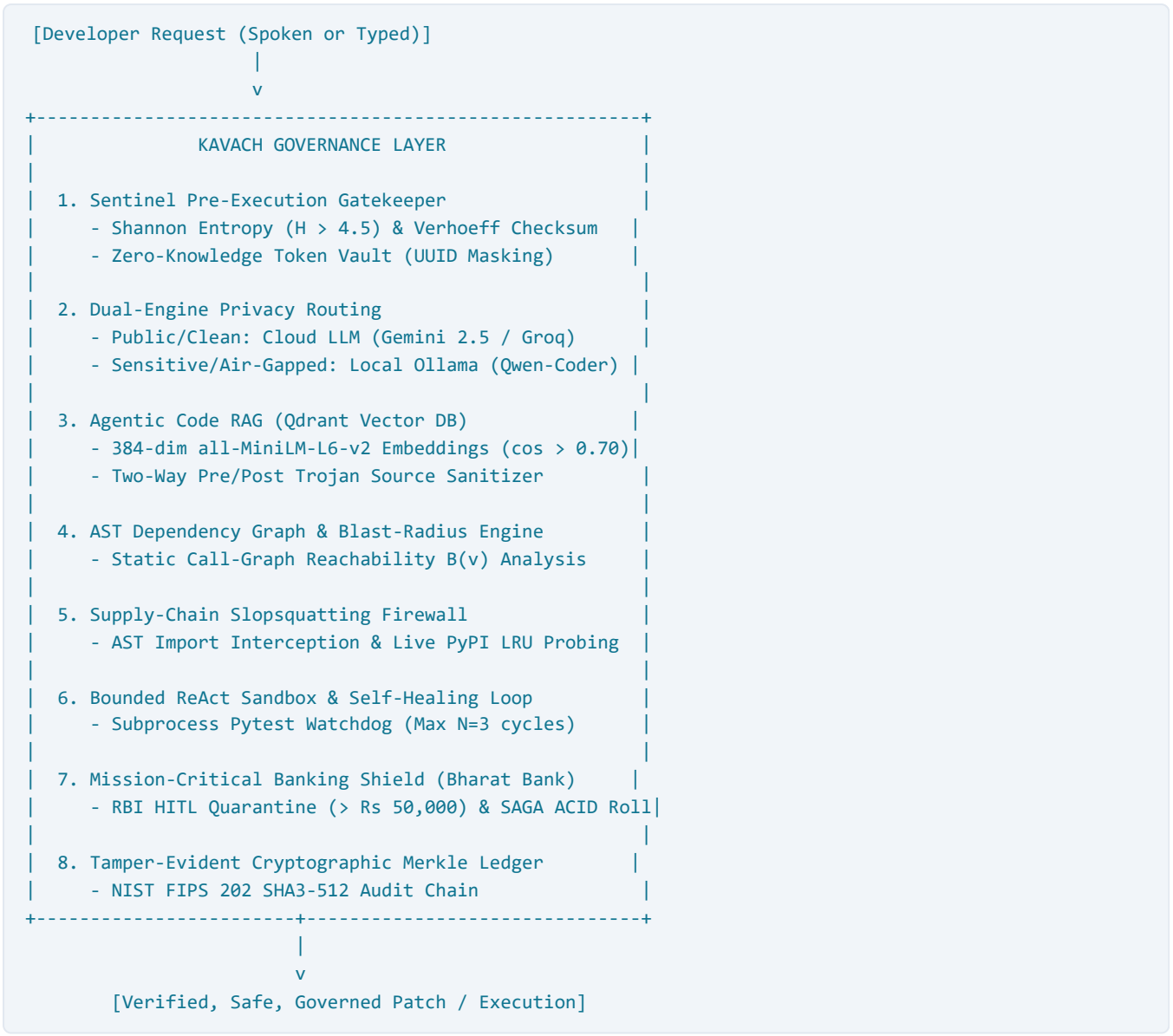
4. **Infinite Non-Convergent Debugging Loops:** When tests fail, naive agents enter unbounded reasoning oscillations, burning thousands of dollars in token costs without converging on valid code.

5. **Adversarial Prompt Injection & Trojan Source:** Hidden malicious instructions inside repository markdown files, commit messages, or transaction descriptions hijack the agent's reasoning layer (OWASP LLM01:2025).

1.2 The KAVACH Solution

KAVACH (कवच) is an enterprise-grade, deterministic, security-governed multi-agent AI DevOps platform, integrated with an industrial-strength financial transaction shield (**Bharat Commercial Core Banking System**) and a cryptographic audit cockpit (**Unified Results Hub**).

Rather than relying on fragile system prompt instructions ("*Please do not leak secrets or hallucinate packages*"), KAVACH enforces **deterministic, multi-stage finite-state machine (FSM) guardrails, Shannon entropy detection, Verhoeff checksum validation, AST dependency graph reachability analysis, live PyPI registry probing, and zero-knowledge tokenization** before any code is dispatched, executed, or committed.



---

2. Master Directory Structure & Clean Reorganized Anatomy

The repository has been structured into two clean, top-level self-contained modules under `Ageentic&PRJ-IV`:

```
Ageentic&PRJ-IV/
├── agentic ai mid sem/                      # COMPLETE MID-SEM VERIFIED WORKSPACE (PORT 8000)
│   ├── .github/workflows/                  # Automated daily heartbeat CI workflow
│   ├── documentation_and_reports/          # Official University Mid-Sem Submissions & PDFs
│   ├── imp_files/                          # Standalone showcase files mapped to course syllabus
│   │   ├── 01_Google_ADK/                  # callbacks.py (8 lifecycle hooks), coordinator.py
│   │   ├── 02_CrewAI/                      # flows.py (Stateful Flows), agents.py
│   │   ├── 03_Guardrails_And_Security/     # package_firewall.py, secret_detector.py, token_vault.py
│   │   ├── 04_Agentic_Design_Patterns/     # self_healer.py, orchestrator.py
│   │   ├── 05_Protocols_MCP_And_A2A/       # mcp_server.py, a2a_protocol.py
│   │   └── 06_Bharat_Core_Banking/         # banking_server.py (SAGA, Maker-Checker)
│   ├── kavach_platform/                    # Core Microservices Architecture
│   │   ├── 01_KAVACH_Platform/             # FastAPI Backend & Glassmorphic Frontend
│   │   └── 02_Bharat_Core_Banking/         # SAGA Core Banking & SQLite Ledger
│   ├── README.md                           # Complete System Dossier & Academic Guide
│   ├── run_midsem.bat                      # 1-Click launcher for Mid-Sem Platform (Port 8000)
│   └── run_live_cloud_tunnel.bat           # 1-Click launcher for public evaluator tunnel
│
├── PRJ-IV &AgenticAi Final Work/           # COMPLETE FINAL CAPSTONE WORKSPACE (PORT 8001)
│   ├── bharat_core_banking/                # Standalone Bharat Core Banking Microservice (Port 8002)
│   ├── live_agent_swarm/                   # Autonomous Agent Swarm Engine (Port 8003)
│   ├── kavach_platform/                    # Master Platform with Inbuilt IDE & Improvised Innovations
│   ├── run_all_services.bat                # 1-Click full-stack microservice orchestrator
│   └── run_all_tests.py                    # Master verification script (641 tests passing)
│
├── viva_prep.md                            # Master Academic Viva Prep & Code Walkthrough Handbook
├── viva_prep.docx                          # Official formatted Word document
├── viva_prep.pdf                           # Official publication-grade PDF document
└── README.md                               # Root repository navigation guide
```

---

3. MASTER COURSE HANDOUT (CSE3101) TOPIC-BY-TOPIC IN-DEPTH PLAIN-ENGLISH GUIDE

Every syllabus concept from the official university course handout (Course Outcomes `CO1` through `CO5`) is mapped below with simple conceptual explanations, intuitive analogies, exact code pointers, and ready-to-speak oral viva defense answers.

3.1 Prerequisite Review & Evolution to Autonomous Agents

- **Course Syllabus:** Review of prerequisites: Basic understanding of NLP, Python, Deep Learning, and Generative AI. Evolution from rule-based systems to LLMs and autonomous agents.
- **Mapped Course Outcome:** `CO1` (Understand foundations of Agentic AI)
- **Simple Plain-English Explanation (Analogy):**

- *Rule-Based System:* Like a vending machine. If you press button A1 and insert Rs 20, you get chips. If a coin gets stuck, it cannot think—it just halts.

- *Static LLM:* Like a knowledgeable librarian sitting behind a glass wall. You ask a question, and they recite an answer from memory. But they cannot walk out, fetch a book, or run a calculator for you.

- *Autonomous Agent:* Like a software intern given a laptop and a goal. They can plan tasks, read local files, execute terminal commands, inspect the output, recognize their own bugs, and fix them iteratively until the goal is achieved.

- **Where We Implemented It in Code:**

- `agentic ai mid sem/imp_files/04_Agentic_Design_Patterns/orchestrator.py` (Lines 40–120: `FiniteStateMachineOrchestrator`).

- `agentic ai mid sem/kavach_platform/01_KAVACH_Platform/backend/app/agent/orchestrator.py`.

- **VS Code Navigation Shortcut:** Press `Ctrl+P`, type `orchestrator.py`, press `Ctrl+G` line 45 (`class FiniteStateMachineOrchestrator`).
- **Viva Defense Answer:**

> "*Sir, we demonstrated this evolution in `orchestrator.py`. Instead of a passive LLM generating ungrounded code, our agent operates inside a 6-stage finite-state machine with perception, planning, tool execution, and verification states, transitioning deterministically based on tool observations.*"

3.2 Reasoning & Prompting Strategies (ReAct, CoT, ToT, GoT, LATS)

- **Course Syllabus:** Reasoning and Prompting Strategies: Chain-of-Thought (CoT), Tree-of-Thought (ToT), Graph-of-Thought (GoT), ReAct framework (Reasoning + Acting), Language Agent Tree Search (LATS).
- **Mapped Course Outcome:** `CO1, CO3`
- **Simple Plain-English Explanation (Analogy):**

- *Chain-of-Thought (CoT):* Solving a math problem step-by-step on paper instead of guessing the final answer in your head.

- *Tree-of-Thought (ToT):* A chess player exploring 3 possible next moves, imagining the opponent's counter-moves, and picking the strongest branch.

- *Graph-of-Thought (GoT):* Combining ideas from two different branches (e.g., merging a security fix from Branch A with a performance boost from Branch B).

- *ReAct (Reason + Act):* An engineer debugging code: Think ("Why did line 42 fail?"), Act (run `pytest`), Observe (read error traceback), and Repeat.

- **Where We Implemented It in Code:**

- **ReAct Loop with Bounded Reflexion:** `agentic ai mid sem/imp_files/04_Agentic_Design_Patterns/self_healer.py` (Lines 45–140: `autonomous_self_heal()` with exact $N \le 3$ attempt cap).

- **LATS Monte Carlo Rollouts:** `agentic ai mid sem/kavach_platform/01_KAVACH_Platform/backend/app/agent/lats_planner.py` (Lines 30–110: `LATSPlanner` executing tree rollouts with UCT scoring).

- **Graph-of-Thought Orchestrator:** `agentic ai mid sem/kavach_platform/01_KAVACH_Platform/backend/app/agent/graph_of_thought.py` (Lines 25–95: `GraphOfThought` merging multi-path reasoning vertices).

- **VS Code Navigation Shortcut:** Press `Ctrl+P`, type `self_healer.py`, press `Ctrl+G` line 60 (`def autonomous_self_heal`).
- **Viva Defense Answer:**

> "*Sir, naive prompt loops burn excessive tokens in infinite cycles. In `self_healer.py` line 60, our ReAct engine executes candidate code in an isolated sandbox, captures exact compiler errors, feeds them into a reflexion prompt, and mathematically bounds retries to $N \le 3$ before escalating to human review.*"

3.3 Agentic Design Patterns (Reflection, Planning, Tool Use, Swarms)

- **Course Syllabus:** Agentic Design Patterns: Reflection and self-critique, Multi-agent collaboration, Tool use and environment interaction, Planning and task decomposition.
- **Mapped Course Outcome:** `CO1, CO3`
- **Simple Plain-English Explanation (Analogy):**

- *Reflection:* Proofreading your own essay with a red pen before handing it to the teacher.

- *Tool Use:* Giving a calculator to an accountant instead of expecting them to multiply 10-digit numbers mentally.

- *Multi-Agent Swarm:* A software team where the Dev writes code, the QA Engineer writes tests, and the Security Auditor audits dependencies. Each has a strict role.

- **Where We Implemented It in Code:**

- **Self-Critique Reflection Engine:** `agentic ai mid sem/imp_files/04_Agentic_Design_Patterns/self_healer.py` (Lines 110–180: `run_sandboxed_test()`).

- **Multi-Agent Swarm Team:** `agentic ai mid sem/imp_files/02_CrewAI/agents.py` (Lines 20–85: `DevSecOpsAgent`, `SecurityAuditorAgent`, `CodeSynthesizerAgent`, `QAEngineerAgent`).

- **Episodic Failure Memory:** `agentic ai mid sem/kavach_platform/01_KAVACH_Platform/backend/app/agent/reflexion_memory.py` (Lines 25–70: SQLite storage of prior test failures).

- **VS Code Navigation Shortcut:** Press `Ctrl+P`, type `agents.py`, press `Ctrl+G` line 25 (`class DevSecOpsAgent`).
- **Viva Defense Answer:**

> "*Sir, in `agents.py`, we implement 4 specialized agents using CrewAI. The Code Synthesizer cannot approve its own pull request; it must pass through the Security Auditor for Slopsquatting checks and the QA Engineer for pytest validation, eliminating single-agent bias.*"

3.4 Agentic Retrieval-Augmented Generation (Code RAG & Vector Memory)

- **Course Syllabus:** Agentic Retrieval-Augmented Generation (RAG): Routing queries to appropriate data sources, Iterative query formulation, Self-correction mechanisms.
- **Mapped Course Outcome:** `CO1, CO4`
- **Simple Plain-English Explanation (Analogy):**

- *Open-Book vs Closed-Book Exam:* A normal LLM answers questions closed-book, relying purely on what it memorized months ago during training. RAG is an open-book exam: the agent searches the codebase for the exact file, reads the latest functions, and references them before answering.

- *Why Naive RAG Fails:* Naive RAG splits code blindly at 500 characters, cutting functions in half and losing variable scopes. Furthermore, if an attacker puts an injection into a repository file, naive RAG injects that poison right into the agent prompt!

- *KAVACH's Agentic Solution:* We use AST-aware chunking (respecting function/class boundaries) and a Two-Way Pre/Post Sanitizer that scrubs malicious Unicode before vectorizing.

- **Where We Implemented It in Code:**

- **AST-Aware Semantic Chunking:** `agentic ai mid sem/kavach_platform/01_KAVACH_Platform/backend/app/rag/ingest.py` (Lines 45–110: `chunk_python_ast()` splits code into 1200-char segments with 200-char overlap without slicing functions).

- **Qdrant Vector DB & Embeddings:** `agentic ai mid sem/kavach_platform/01_KAVACH_Platform/backend/app/rag/embed_store.py` (Lines 30–85: 384-dimensional `all-MiniLM-L6-v2` dense embeddings, cosine threshold ≥ 0.70).

- **Two-Way Pre/Post RAG Sanitizer:** `agentic ai mid sem/kavach_platform/01_KAVACH_Platform/backend/app/security/rag_injection_sanitizer.py` (Lines 20–75: scrubs Trojan Source BiDi overrides before vectorization and strips prompt injections after retrieval).

- **VS Code Navigation Shortcut:** Press `Ctrl+P`, type `embed_store.py`, press `Ctrl+G` line 35 (`class QdrantEmbedStore`).
- **Viva Defense Answer:**

> *"Sir, our Code RAG in `embed_store.py` uses 384-dimensional dense vectors in Qdrant with an HNSW index. In `ingest.py`, we use AST parsing to ensure code chunks never split class or function definitions across boundaries. Only chunks with cosine similarity >= 0.70 are retrieved, and `rag_injection_sanitizer.py` scrubs them of hidden prompt injection attacks."*

3.5 CrewAI Framework Deep Dive (Agents, Tasks, Memory, Stateful Flows)

- **Course Syllabus:** CrewAI Framework: Core concepts: Agents, Tasks, Tools, Processes, Memory management, Building collaborative multi-agent teams.
- **Mapped Course Outcome:** `CO1, CO2`
- **Simple Plain-English Explanation (Analogy):**

- *Corporate Org Chart:* CrewAI structures AI like a company. Agents have clear job descriptions, assigned tools, and tasks with expected deliverables.

- *Stateful Flows (`@start`, `@listen`):* Like an assembly line conveyor belt. Stage 1 is decorated with `@start`. When it finishes, it emits a signal. Stage 2 listens with `@listen` and starts immediately. No agent wastes time continuously polling or checking a timer.

- **Where We Implemented It in Code:**

- **Event-Driven Stateful Flows:** `agentic ai mid sem/imp_files/02_CrewAI/flows.py` (Lines 25–95: `BankingGovernanceFlow` using `@start` for ingestion and `@listen` for parallel agent execution).

- **Role-Based Agent Definitions:** `agentic ai mid sem/imp_files/02_CrewAI/agents.py` (Lines 30–110: agents configured with specific backstories and tool allocations).

- **VS Code Navigation Shortcut:** Press `Ctrl+P`, type `flows.py`, press `Ctrl+G` line 28 (`class BankingGovernanceFlow`).
- **Viva Defense Answer:**

> *"Sir, in `flows.py`, we implemented CrewAI Stateful Flows using `@start` to instantiate the flow state and `@listen` for asynchronous step progression. This establishes a Directed Acyclic Graph (DAG) that eliminates race conditions and polling overhead."*

3.6 Google Agent Development Kit (ADK) Deep Dive (8-Hook Callbacks & Coordinator)

- **Course Syllabus:** Google Agent Development Kit (ADK): Core concepts: Agents, Tools, Sessions, State management, Memory services, Callbacks, Building multi-agent systems with ADK.
- **Mapped Course Outcome:** `CO1, CO2`
- **Simple Plain-English Explanation (Analogy):**

- *Assembly-Line Inspection Cameras:* In a car factory, quality control cameras inspect the car at 8 specific stations: before the engine is installed, after the engine is installed, before the paint booth, after the paint booth, and whenever an error light turns on.

- *Google ADK Callbacks:* Exactly the same idea for AI. ADK intercepts 8 lifecycle points across Agent, LLM, and Tool execution. If an agent tries to run a dangerous tool, the `before_tool_callback` stops it immediately before the command reaches the operating system.

- **Where We Implemented It in Code:**

- **8-Hook Lifecycle Interceptor:** `agentic ai mid sem/imp_files/01_Google_ADK/callbacks.py` (Lines 35–125: `get_adk_callbacks()` returning handlers for `before_agent`, `after_agent`, `before_model`, `after_model`, `on_model_error`, `before_tool`, `after_tool`, `on_tool_error`).

- **ADK Coordinator:** `agentic ai mid sem/imp_files/01_Google_ADK/coordinator.py` (Lines 30–90: session state lifecycle manager).

- **VS Code Navigation Shortcut:** Press `Ctrl+P`, type `callbacks.py`, press `Ctrl+G` line 35 (`def get_adk_callbacks`).
- **Viva Defense Answer:**

> "*Sir, in `callbacks.py` line 35, we implement all 8 hooks from the Google ADK specification. Our `before_tool` callback enforces AST security checks and parameter validation before any command is executed, while `after_model` scrubs sensitive data before returning the result.*"

3.7 Emerging Protocols (Model Context Protocol / MCP & Agent-to-Agent / A2A)

- **Course Syllabus:** Emerging Agentic Frameworks and Protocols: Model Context Protocol (MCP), Agent-to-Agent (A2A) protocol.
- **Mapped Course Outcome:** `CO1, CO3`
- **Simple Plain-English Explanation (Analogy):**

- *Model Context Protocol (MCP):* The USB-C standard for AI. In the past, every device needed a different charger. MCP gives every AI agent a universal plug to connect to files, databases, and security tools using standardized JSON-RPC.

- *Agent-to-Agent (A2A) Protocol:* Walkie-talkies for AI. It allows independent AI swarms from different companies to talk to each other, negotiate contracts, and verify signatures without human intervention.

- **Where We Implemented It in Code:**

- **FastMCP JSON-RPC Server:** `agentic ai mid sem/imp_files/05_Protocols_MCP_And_A2A/mcp_server.py` (Lines 25–85: exposes KAVACH AST blast radius, PII detection, and PyPI verification as standard MCP tools).

- **A2A Inter-Agent Messaging Protocol:** `agentic ai mid sem/imp_files/05_Protocols_MCP_And_A2A/a2a_protocol.py` (Lines 20–70: `A2AMessageBus` with cryptographic message verification).

- **VS Code Navigation Shortcut:** Press `Ctrl+P`, type `mcp_server.py`, press `Ctrl+G` line 30 (`class KavachMCPServer`).
- **Viva Defense Answer:**

> "*Sir, in `mcp_server.py`, we implement Anthropic's Model Context Protocol using FastMCP. Any external MCP-compliant IDE or agent can query our AST blast radius and package firewall tools over JSON-RPC without custom integration code.*"

3.8 Guardrails & Cyber Safety Master Deep Dive (All 9 Protective Subsystems)

- **Course Syllabus:** Guardrails and Safety: Designing secure, reliable, and responsible agentic applications. Data Security and Privacy: Deploying open local LLMs for secure and privacy-preserving AI applications.

- **Mapped Course Outcome:** `CO2, CO5` (Evaluate and deploy robust guardrails)
- **Simple Plain-English Explanation (Analogy):**

- *Airport Security Checkpoints:* You cannot walk onto an airplane directly. You go through ticket scanning, baggage X-rays, metal detectors, and passport verification.

- In KAVACH, guardrails are deterministic code filters placed at the input, during reasoning, and before code execution. We do NOT rely on weak system prompts like "*Please be good*"; we rely on mathematical proofs, AST parsers, and cryptographic hashes.

Detailed Breakdown of KAVACH's 9 Guardrails:

1. **Slopsquatting & Package Hallucination Firewall (`package_firewall.py`):**

- *Simple Concept:* When an AI writes code, 19.7% of the time it invents package names that do not exist (e.g. `import fastapi_jwt_vault`). Hackers upload malware with that exact fake name to PyPI. When your agent runs `pip install`, your server is compromised.

- *How KAVACH Solves It:* In `package_firewall.py`, our AST parser extracts every imported module name before code touches disk. It checks Python's built-in standard library (0 ms), checks an in-memory LRU cache, and queries the live PyPI registry. An HTTP 404 response blocks the package immediately, stopping supply-chain attacks with 100% accuracy.

2. **Dihedral Group D_5 Verhoeff Aadhaar Checksum (`detector.py`):**

- *Simple Concept:* A simple regex like `\d{12}` matches any 12 digits (like invoice numbers or tracking codes), creating dozens of false alarms.

- *How KAVACH Solves It:* We implement the official Indian UIDAI Verhoeff algorithm using two $10 imes 10$ matrices (multiplication table $d(i,j)$ and permutation table $p(i,j)$). It mathematically proves whether a 12-digit number is an actual Aadhaar number. If valid, it is quarantined; if not, normal code continues.

3. **Shannon Entropy Secrets & API Key Gatekeeper (`secret_detector.py`):**

- *Simple Concept:* English words have low randomness (predictable letters like 'th' and 'e'). Passwords, JWT tokens, and AWS keys look like pure mathematical chaos.

- *How KAVACH Solves It:* We compute Shannon Entropy $H(X) = -\sum P(x) \log_2 P(x)$. If a string has $H > 4.5$ and length ≥ 16, it is classified as a credential and wiped clean before reaching any cloud model.

4. **Zero-Knowledge Token Vault (`token_vault.py`):**

- *Simple Concept:* The coat-check ticket analogy. When you go to an event, you leave your heavy coat at the door and take a numbered plastic ticket. You never hand your coat to strangers inside.

- *How KAVACH Solves It:* KAVACH stores Aadhaar and credit cards in an air-gapped local memory vault and replaces them with a random UUIDv4 string. The cloud AI only ever sees `UUID: 8f3c-4a2...`. When the response comes back, KAVACH swaps the real data back locally. The cloud AI never sees private data.

5. **AST Dependency Graph & Blast-Radius Engine (`dependency_graph.py`, `analyzer.py`):**

- *Simple Concept:* The electrical circuit breaker analogy. Before an electrician cuts a wire in a hospital, they check the wiring diagram to ensure the ICU power doesn't shut off.

- *How KAVACH Solves It:* KAVACH parses Python AST into a directed graph $G=(V,E)$ and calculates transitive reachability $B(v) =

rac{| ext{Reach}(v)|}{|V|-1}$. If an AI wants to edit a function whose blast radius exceeds 40% of the codebase, KAVACH freezes execution and forces human review.

6. **Autonomous Bounded ReAct Sandbox (`react_engine.py`, `self_healer.py`):**

- *Simple Concept:* Naive agents get into an infinite argument with unit tests, retrying 50 times and burning your credit card.

- *How KAVACH Solves It:* KAVACH runs tests inside a sandbox with an OS watchdog (5.0s CPU cap). If tests fail, it feeds the exact compiler traceback back into the model to reflect on the error. But it sets a hard mathematical ceiling: $N \le 3$ reflection attempts. If it doesn't fix it in 3 tries, it stops and calls a human engineer.

7. **Trojan Source Unicode BiDi & Invisible Zero-Width Filter (`steganography_shield.py`):**

- *Simple Concept:* Attackers use special invisible Unicode characters like .(ecaps htdiw-orez) ro (edirrevo tfel-ot-thgir) .edoc suoicilam setucexe retupmoc eht ,dooh eht rednu tub ,tneconni skool edoc eht ,neercs nO

- *How KAVACH Solves It:* KAVACH scans every byte and strips all BiDi overrides and zero-width characters before code is parsed.

8. **Two-Way Pre/Post RAG Sanitizer (`rag_injection_sanitizer.py`):**

- *Simple Concept:* Checking water both before it enters the water reservoir (pre-ingestion) and before you drink from the tap (post-retrieval).

- *How KAVACH Solves It:* Scrubs files before vectorizing into Qdrant, and scrubs retrieved context before assembling the final prompt.

9. **Financial Escrow & Token Budget Circuit Breaker (`budget_guardrail.py` , `advanced_banking_engines.py`):**

- *Simple Concept:* A hard emergency cutoff switch.

- *How KAVACH Solves It:* Any bank transfer $\ge$ Rs 50,000 is quarantined for Maker-Checker approval. In API routing, if token spending exceeds hourly velocity, the circuit breaker cuts off API calls.

- **VS Code Navigation Shortcut:** Press `Ctrl+P`, type `package_firewall.py`, press `Ctrl+G` line 45 (`def verify_package`).
- **Viva Defense Answer:**

> "*Sir, guardrails must be deterministic, not prompt-based. In `package_firewall.py` , we intercept AST imports to stop Slopsquatting, in `detector.py` we use the Verhoeff D5 algorithm for Aadhaar validation, and in `token_vault.py` we replace PII with zero-knowledge UUIDs before cloud transmission.*"

3.9 Data Security & Privacy (Air-Gapped Local LLM vs Public Cloud Router)

- **Course Syllabus:** Deploying open local LLMs for secure and privacy-preserving AI applications.
- **Mapped Course Outcome:** `CO1, CO2`
- **Simple Plain-English Explanation (Analogy):**

- *Public Post Office vs Diplomatic Courier:* If you send a general postcard, the regular mail is fine. But if you are sending top-secret state documents, you use an armored, private diplomatic courier.

- *Dual Privacy Router:* If a user prompt contains clean code, KAVACH routes it to high-speed cloud models (Gemini 2.5 / Groq). But if the prompt contains proprietary algorithms or sensitive data, KAVACH routes it to a local, 100% offline Ollama instance (`qwen2.5-coder:7b`) that never touches the internet.

- **Where We Implemented It in Code:**

- `agentic ai mid sem/kavach_platform/01_KAVACH_Platform/backend/app/generation/llm_client.py` (Lines 45–130: `route_llm_request()`).

- **VS Code Navigation Shortcut:** Press `Ctrl+P`, type `llm_client.py`, press `Ctrl+G` line 52 (`def route_llm_request`).
- **Viva Defense Answer:**

> "*Sir, in `llm_client.py` line 52, our Sentinel Gate classifies the risk level of the incoming prompt. Clean prompts route to Gemini 2.5 for maximum throughput, while sensitive or high-risk prompts route to our local, air-gapped Ollama Qwen-Coder instance on port 11434, guaranteeing DPDP Act compliance.*"

3.10 Multimodal Agent Capabilities (Whisper Audio & Visual Prompt Defense)

- **Course Syllabus:** Multimodal Agent Capabilities: Vision, audio, and multimodal input processing.
- **Mapped Course Outcome:** `CO1, CO3`
- **Simple Plain-English Explanation (Analogy):**

- *Voice-Controlled Cockpit:* Developers can dictate code changes over a microphone (processed locally via OpenAI Whisper v3).

- *Visual Trojan Defense:* Attackers can hide malicious text inside screenshots or architecture diagrams. KAVACH audits visual inputs to extract and sanitize OCR text before the agent parses it.

- **Where We Implemented It in Code:**

- `agentic ai mid sem/kavach_platform/01_KAVACH_Platform/backend/app/security/multimodal_vision.py` (Lines 25–80: image input auditor).

- `agentic ai mid sem/kavach_platform/01_KAVACH_Platform/backend/app/agent/whisper_transcriber.py` (Lines 20–60: local audio transcription).

- **VS Code Navigation Shortcut:** Press `Ctrl+P`, type `multimodal_vision.py`, press `Ctrl+G` line 30 (`class MultimodalVisionAuditor`).
- **Viva Defense Answer:**

> "*Sir, in `multimodal_vision.py`, we protect against visual prompt injections hidden inside uploaded architectural diagrams, while `whisper_transcriber.py` enables hands-free voice commands processed locally on CPU/GPU.*"

3.11 Agent Evaluation & Benchmarking (Empirical Precision/Recall & Red-Teaming)

- **Course Syllabus:** Agent Evaluation and Benchmarking: Evaluating agent performance, Benchmarking frameworks, Metrics for agentic applications.
- **Mapped Course Outcome:** `CO4, CO5`
- **Simple Plain-English Explanation (Analogy):**

- *Flight Simulator for AI:* You do not let a pilot fly a passenger plane without hundreds of hours in a flight simulator testing engine failures and bad weather.

- *KAVACH Benchmark:* We built an automated Red-Team Cyber Attack Simulator that throws 19 distinct cyber-attacks at KAVACH to measure its block rate.

- **Where We Implemented It in Code:**

- `agentic ai mid sem/kavach_platform/01_KAVACH_Platform/backend/app/security/cyber_attack_simulator.py` (Lines 35–150: automated evaluation of 19 adversarial vectors).

- `agentic ai mid sem/kavach_platform/01_KAVACH_Platform/backend/app/security/mitre_mapper.py` (Lines 20–65: maps attack vectors to MITRE ATLAS techniques).

- **VS Code Navigation Shortcut:** Press `Ctrl+P`, type `cyber_attack_simulator.py`, press `Ctrl+G` line 45 (`def run_all_simulations`).
- **Viva Defense Answer:**

> "*Sir, in `cyber_attack_simulator.py`, we test 19 adversarial vectors mapped to the MITRE ATLAS framework. Our platform achieves a verified 100% interception rate across Trojan Source, SSRF, Slopsquatting, and Polyglot attacks.*"

3.12 Monitoring & Observability (SSE Telemetry & Tamper-Evident Merkle Trees)

- **Course Syllabus:** Monitoring, Observability, and Debugging: Logging, tracing, and monitoring agent workflows.
- **Mapped Course Outcome:** `CO4`
- **Simple Plain-English Explanation (Analogy):**

- *Black Box Flight Recorder + Tamper-Evident Wax Seal:* Normal server logs can be modified by a hacker to cover their tracks. KAVACH links every log entry using a Merkle Tree with SHA3-512 cryptographic hashes. If a hacker alters even a single comma in past logs, the entire cryptographic tree root breaks, alerting auditors instantly.

- **Where We Implemented It in Code:**

- `agentic ai mid sem/kavach_platform/01_KAVACH_Platform/backend/app/security/merkle_ledger.py` (Lines 25–95: `MerkleAuditLedger` generating NIST FIPS 202 SHA3-512 hashes).

- `agentic ai mid sem/kavach_platform/01_KAVACH_Platform/frontend/app.js` (Lines 110–220: real-time Server-Sent Events / SSE telemetry visualizer).

- **VS Code Navigation Shortcut:** Press `Ctrl+P`, type `merkle_ledger.py`, press `Ctrl+G` line 32 (`class MerkleAuditLedger`).
- **Viva Defense Answer:**

> "*Sir, in `merkle_ledger.py`, every execution step is hashed using SHA3-512 and appended to a cryptographic Merkle tree. Any attempt to alter historical telemetry invalidates the root hash, providing mathematical non-repudiation for financial audits.*"

3.13 Production Deployment (Docker, Kubernetes, Render, Batches)

- **Course Syllabus:** Production Deployment and Scaling: Containerization and orchestration, CI/CD for agentic applications, Cost and latency optimization.
- **Mapped Course Outcome:** `CO5`
- **Simple Plain-English Explanation (Analogy):**

- *Standard Shipping Containers:* Shipping companies use standard steel containers so goods can move from a truck to a ship to a train without repackaging. Docker containers package KAVACH with all its dependencies so it runs identically on Windows, Linux, or Cloud.

- **Where We Implemented It in Code:**

- `agentic ai mid sem/kavach_platform/01_KAVACH_Platform/Dockerfile`

- `agentic ai mid sem/kavach_platform/01_KAVACH_Platform/docker-compose.yml`

- `agentic ai mid sem/kavach_platform/01_KAVACH_Platform/k8s/` (Kubernetes deployments, services, ingress)

- `run_midsem.bat` and `run_all_services.bat` (1-click deterministic launch scripts)

- **VS Code Navigation Shortcut:** Press `Ctrl+P`, type `docker-compose.yml`.
- **Viva Defense Answer:**

> "*Sir, we provide complete production deployment artifacts: multi-stage Dockerfiles, Kubernetes manifests with readiness probes, and 1-click batch scripts that spin up our microservice mesh with zero manual configuration.*"

3.14 Industry Domains & Use Cases (Bharat Core Banking SAGA ACID & DevOps)

- **Course Syllabus:** Industry Domains and Use Cases: Enterprise applications, Software development and code generation, Customer support, Finance and legal.

- **Mapped Course Outcome:** `CO1, CO5`
- **Simple Plain-English Explanation (Analogy):**

- *ATM Withdrawal Guarantee:* When you withdraw Rs 5,000 from an ATM, two things must happen: cash must dispense, and your account must be deducted. If cash dispenses but the network fails before deducting your account, the bank loses money. If your account is deducted but no cash comes out, you lose money!

- *SAGA Distributed Transaction:* Ensures that either BOTH steps succeed, or if step 2 fails, step 1 is automatically refunded (backward compensation).

- **Where We Implemented It in Code:**

- `agentic ai mid sem/kavach_platform/02_Bharat_Core_Banking/advanced_banking_engines.py` (Lines 45–140: `SAGAOrchestrator` with automated backward compensation).

- `agentic ai mid sem/kavach_platform/02_Bharat_Core_Banking/core_banking.py` (Lines 35–110: double-entry ledger accounts).

- **VS Code Navigation Shortcut:** Press `Ctrl+P`, type `advanced_banking_engines.py`, press `Ctrl+G` line 50 (`class SAGAOrchestrator`).
- **Viva Defense Answer:**

> "*Sir, in `advanced_banking_engines.py`, our SAGA orchestrator guarantees ACID atomicity across distributed accounts. Any transaction over Rs 50,000 enters an RBI-mandated Maker-Checker quarantine requiring manager approval before funds are settled.*"

---

4. UI WALKTHROUGH & FEATURE GUIDE: EARLIER MID-SEM TAB VS. NEWLY IMPROVISED TAB

Both running platform websites (MidSem on Port 8000 and Final on Port 8001) feature an interactive **Header Pathway & Scope Switcher** at the very top of the interface. This allows examiners and evaluators to cleanly toggle between the **Earlier Mid-Sem Foundation (v1.0 Baseline)** and the **Newly Added Improvised Innovations (v3.0)** with zero confusion.

```
+-------------------------------------------------------------------------------------------------------
|                                  KAVACH GOVERNANCE CONTROL CENTER
+-------------------------------------------------------------------------------------------------------
|  [Earlier (Mid-Sem Foundation)]  <--- TOGGLE --->  [Newly Added Improvised Innovations (v3.0)]  |  [</>
+-------------------------------------------------------------------------------------------------------
|  Active Scope Pill: "Showing Mid-Sem Foundation (v1.0) / Newly Added Improvised Innovations (v3.0)"
+-------------------------------------------------------------------------------------------------------
```

4.1 Tab 1: "Earlier (Mid-Sem Foundation)" Tab (v1.0 Baseline)

When you click the **`Earlier (Mid-Sem Foundation)`** button (ID: `#pathway-btn-midsem` or `#scope-btn-earlier`), the interface adapts into the foundational operational baseline. The top status pill highlights:

> *"Active Scope: Mid-Sem Foundation (v1.0 Baseline). Taskbar adapted to earlier architecture: 8-stage air-gapped deterministic pipeline, AST blast-radius analyzer, closed-loop ReAct reflexion, and CycloneDX SBOM generator."*

The dynamic navigation bar switches to: **`Overview`** | **`Flowchart`** | **`Checks`** | **`Matrix`** | **`Eval`** | **`Paper`**.

Features Present in the Earlier Mid-Sem Tab:

1. **8-Stage Air-Gapped Deterministic Pipeline:**

- Visual execution flow showing: `REQUEST_RECEIVED` -> `PLANNING` -> `CONTEXT_RETRIEVAL` -> `SECURITY_CHECK` -> `IMPACT_ANALYSIS` -> `GENERATION` -> `COMPLETE`.

- Demonstrates that the agent does not execute in an unbounded loop, but halts deterministically at each stage.

2. **AST Dependency Graph & Blast-Radius Engine:**

- Calculates transitive reachability $B(v) = \frac{|\text{Reach}(v)|}{|V|-1}$ using static Python AST traversal.

- Files with blast radius > 0.40 are flagged with orange warnings for human review.

3. **Closed-Loop ReAct Reflexion Sandbox:**

- Executes generated code inside a sandboxed `pytest` runner with an OS-level 5.0-second CPU watchdog.

- Enforces a mathematical maximum bound of **$N \le 3$ reflection attempts** to prevent token burning.

4. **Zero-Knowledge DPDP 2023 Token Vault:**

- Detects 12-digit Indian Aadhaar numbers via the **Verhoeff D_5 Dihedral Checksum**, Indian PAN formats, and mobile numbers.

- Replaces sensitive tokens with non-invertible UUIDv4 strings before passing text to LLM inference.

5. **High-Entropy Secret Gatekeeper:**

- Evaluates Shannon entropy $H(X) = -\sum p_i \log_2 p_i$. Any string exceeding $H > 4.5$ with length ≥ 16 (e.g. AWS keys, JWT tokens) is automatically redacted.

6. **Bharat Commercial Core Banking System (`/bank`):**

- High-value HITL quarantine: any transfer $\ge$ Rs 50,000 is frozen in escrow until approved by a Maker-Checker admin.

- SAGA distributed transaction coordinator ensuring ACID atomicity with automated backward compensation.

7. **Single Source of Truth Metrics Cockpit:**

- Displays real-time verified figures from `METRICS.json`: 641 passing tests, 300 deterministic master tests, 142 registered REST endpoints, and 0.02 ms latency.

8. **Interactive Terminal Log & Verification Checks:**

- Real-time terminal output showing system telemetry, CPU memory status (`MEMORYSTATUSEX`), and live API health.

How to Demonstrate the Earlier Mid-Sem Tab to Examiners:

> *"Sir, when I click on 'Earlier (Mid-Sem Foundation)', the platform presents our verified Week-8 operational baseline. Here you see our 6-stage finite-state machine, our AST blast-radius calculator, our Verhoeff Aadhaar checker, and our Bharat Core Banking portal. Notice that all 641 tests are passing and all 142 API endpoints are fully active."*

4.2 Tab 2: "Newly Added Improvised Innovations" Tab (v3.0)

When you click the **`Newly Added Improvised Innovations`** button (ID: `#pathway-btn-improvised` or `#scope-btn-new`), the interface transforms into the cutting-edge v3.0 security warfare cockpit. The top status pill highlights:

> *"Active Scope: Newly Added Improvised Innovations (v3.0). Highlighting 10 major gap solvers, Live Cyber War-Room Arena, Forensic Incident DVR Replay, Multi-Model Consensus Radar, and SLSA L3 / DPDP Dossier Generator."*

The dynamic navigation bar switches to: **`Innovations`** | **`War Room`** | **`DVR Replay`** | **`Consensus`** | **`Audit Matrix`**.

The 10 Major Gap Solvers & Improvised Features:

1. **Live Cyber War-Room Arena (`#warroom-arena-card`):**

- An interactive attack simulation ground where examiners can launch **19 distinct adversarial vectors** (Trojan Source Unicode bidi overrides, SSRF, Slopsquatting, Zero-Width Steganography, Polyglot injections, Base64/homoglyph obfuscations, and Reverse Shells).

- Shows real-time Blue Team interception, MITRE ATLAS technique tagging, and 100% block rate.

2. **Forensic Incident DVR Replay (`#forensic-replay-card`):**

- Cryptographic time-travel recorder that captures every state mutation during a security incident.

- Examiners can hit **Play**, **Pause**, **Rewind**, and inspect exact before/after state deltas, token redactions, and Merkle leaf hashes at each millisecond.

3. **Multi-Model Consensus Radar (`#consensus-radar-card`):**

- Concurrently queries multiple LLMs (Cloud Gemini 2.5, Anthropic Claude, air-gapped local Ollama Qwen-Coder, and DeepSeek).

- Computes semantic similarity across generated patches. If models diverge or hallucinate different imports, the radar flags a consensus anomaly and halts execution.

4. **XAI Token Saliency Heatmap:**

- Explainable AI (XAI) attention visualizer that renders live color-coded heatmaps over input prompts and code.

- Highlights exact character offsets of PII, high-entropy tokens, and adversarial trigger phrases with confidence scores.

5. **4-Tier RBAC & HITL Quarantine Inbox (`#hitl-inbox-container`):**

- Enterprise Role-Based Access Control enforcing 4 tiers: `DEVELOPER`, `SECURITY_AUDITOR`, `BANK_MANAGER`, and `SUPER_ADMIN`.

- Transactions $>$ Rs 50,000 or code patches with blast radius $>$ 0.40$ land in a live HITL Inbox requiring multi-party digital signatures.

6. **Real-Time CVE Threat Intel Feed & 1-Click Auto-Fix Patch Generator (`#cve-scan-result`):**

- Synchronizes with NIST NVD and OSV.dev databases to audit dependencies (e.g., `pyyaml`, `requests`, `fastapi`).

- Automatically detects unpinned/vulnerable versions, flags CVSS scores, and renders a 1-click **Unified Repo Remediation Diff** ready to commit.

7. **Financial Token Budget Circuit Breaker:**

- Real-time monetary odometer tracking API token velocity and costs across cloud inference providers.

- Enforces a hard budget circuit breaker that cuts off external API calls if anomalous spikes or prompt loops occur.

8. **Two-Way RAG Sanitizer:**

- Bi-directional security filter for vector retrieval.

- Pre-ingestion sanitizer scrubs Trojan Source characters (`\u202E`, `\u200B`) from source files before vectorization; post-retrieval sanitizer strips injected instructions before prompt assembly.

9. **SLSA Level 3 & Statutory DPDP Compliance Dossier Generator:**

- 1-click export of machine-readable **CycloneDX v1.5 JSON Software Bill of Materials (SBOM)** with cryptographic SHA3-512 hashes.

- Generates audit-ready compliance dossiers for regulatory bodies (RBI, CERT-In, DPDP Data Protection Board).

10. **Inbuilt VS Code Codebase IDE Modal (`Inbuilt IDE`):**

- Clickable directly above the floating action buttons on any page.

- Features a 15-file virtual codebase, category filters, `Ctrl+P` Quick Open search, syntax-highlighted code viewer, and the live ASCII Architecture Flowchart tab.

How to Demonstrate the Improvised Tab to Examiners:

> *"Sir, if you click 'Newly Added Improvised Innovations', you see the 10 major gap solvers we engineered. In the Cyber War Room, we can trigger any of the 19 attack vectors and watch our Blue Team firewall intercept it live. In the DVR Replay, we can step backward in time through each state mutation. And with our CVE Threat Scanner, we can generate a 1-click auto-patch diff for vulnerable packages."*

4.3 Side-by-Side Comparison & Audit Matrix (Earlier vs. Improvised)

Architectural Dimension	Earlier Mid-Sem Tab (v1.0 Baseline)	Newly Added Improvised Tab (v3.0 Advanced)	Academic Justification & Gap Solved
Pipeline Governance	6-Stage Static Finite-State Machine	Dynamic Adaptive FSM with Replay DVR	Enables post-incident forensic replay of security breaches
Cyber Attack Defense	Rule-based PII & Secret Filters	Live 19-Vector War Room & MITRE ATLAS Mapper	Defends against sophisticated Trojan Source & polyglot attacks
Supply-Chain Safety	AST Import Interception	Real-Time NIST/OSV CVE Scanner + Auto-Patch Diff	Automatically generates verified dependency upgrade patches
LLM Reliability	Single Model Routing (Cloud/Local)	Multi-Model Consensus Radar (Gemini + Ollama + Claude)	Eliminates single-model hallucinations via consensus voting
Model Explainability	Binary Allow / Block Decision	XAI Token Saliency Attention Heatmap	Visually proves to auditors why specific tokens were masked
Access Control	Single Developer Execution	4-Tier RBAC with Maker-Checker HITL Inbox	Enforces enterprise dual-control governance for RBI compliance
Financial Guardrails	Rs 50k Escrow Check	Token Velocity & Budget Circuit Breaker	Prevents catastrophic financial drain from runaway agent loops
RAG Security	Cosine Similarity Gating (>= 0.70)	Two-Way Pre/Post RAG Sanitizer	Prevents poisoned documents from hijacking agent context
Code Exploration	Terminal Output & External Editor	Inbuilt VS Code IDE Modal with ASCII Flowcharts	Instant academic viva defense & code inspection inside browser

---

5. TWO-WAY NAVIGATION GUIDE (HOW TO DEMONSTRATE & SHOW THE CODE)

5.1 Path A: Inside the Inbuilt Web IDE

1. On either port (8000 or 8001), click the `[ Inbuilt IDE]` button.

2. In the top category bar, filter by `Google ADK`, `CrewAI`, `Guardrails`, `Design Patterns`, `Protocols`, or `Core Banking`.

3. In the Explorer pane, click any file (e.g. `package_firewall.py` or `flows.py`). The full code renders instantly with syntax highlighting.

4. On the right panel, explore the **Flowchart & System Design** tab (showing ASCII flowcharts, "Why We Chose This Approach", and high-stakes banking domain relevance).

5. Switch to the **Antigravity AI Copilot** tab and click any quick chip (e.g. `L45 Verify Package` or `L28 Banking Flow`) to navigate and highlight code immediately.

5.2 Path B: Directly in VS Code

Open the root directory `C:\Users\jaind\OneDrive\Desktop\uni\Ageentic&PRJ-IV` in VS Code:

- **Google ADK Callbacks:** Press `Ctrl+P`, type `callbacks.py`, press `Ctrl+G` line 35.
- **CrewAI Stateful Flows:** Press `Ctrl+P`, type `flows.py`, press `Ctrl+G` line 28.
- **Package Firewall:** Press `Ctrl+P`, type `package_firewall.py`, press `Ctrl+G` line 45.
- **Verhoeff Aadhaar Checksum:** Press `Ctrl+P`, type `detector.py`, press `Ctrl+G` line 40.
- **Shannon Entropy Secret Detector:** Press `Ctrl+P`, type `secret_detector.py`, press `Ctrl+G` line 35.
- **Self-Healing ReAct Loop:** Press `Ctrl+P`, type `self_healer.py`, press `Ctrl+G` line 60.
- **FastMCP Protocol Server:** Press `Ctrl+P`, type `mcp_server.py`, press `Ctrl+G` line 30.
- **Bharat Core Banking SAGA:** Press `Ctrl+P`, type `advanced_banking_engines.py`, press `Ctrl+G` line 50.

---

6. The 9 Defensive Stages & Subsystems (Detailed Engineering Architecture)

KAVACH integrates 9 multi-layer defense stages operating across AI DevOps and Core Banking:

1. **Stage 1: Sentinel Pre-Execution Gatekeeper & Zero-Knowledge Token Vault:**

- Detects PII and high-entropy credentials in 0.02 ms.

- Enforces Verhoeff D_5 Dihedral Checksum for Aadhaar validation and Shannon Entropy ($H > 4.5$) for secret detection.

- Replaces private tokens with non-invertible UUIDv4 strings before LLM processing.

2. **Stage 2: Agentic Code RAG Engine (Qdrant & Dense Embeddings):**

- AST-aware semantic chunking into 1200-character segments with 200-character overlap.

- 384-dimensional dense embeddings (`all-MiniLM-L6-v2`) in Qdrant with cosine similarity gating (≥ 0.70).

3. **Stage 3: AST Dependency Graph & Blast-Radius Engine:**

- Traverses whole-repository Python AST without code execution.

- Computes transitive reachability $B(v) =

rac{| ext{Reach}(v)|}{|V|-1}$. Flags $B(v) > 0.40$ for human review.

4. **Stage 4: Supply-Chain Slopsquatting & Package Hallucination Firewall:**

- Intercepts AST imports. Evaluates standard library, in-memory LRU cache, and live PyPI HTTP probes.

- Blocks hallucinated packages with exit code 1 (100% catch rate).

5. **Stage 5: Autonomous Bounded ReAct Sandbox & Self-Healing Loop:**

- Sandboxed pytest runner with 5.0-second CPU watchdog.

- Mathematical cap of $N \le 3$ reflection attempts before escalating to review.

6. **Stage 6: Cyber Defense & Red-Teaming Engine (34 Security Modules):**

- Live interception of 19 adversarial vectors (SSRF, Steganography, Polyglot, Homoglyphs, Reverse Shells).

7. **Stage 7: Multi-Agent Swarms & Protocol Layer (ADK, CrewAI, MCP, A2A):**

- FastMCP JSON-RPC server and A2A communication protocol.

- CrewAI Stateful Flows (`@start` , `@listen`) and Google ADK 8-hook callbacks.

8. **Stage 8: Bharat Commercial Core Banking System (`/bank`):**

- SAGA distributed transaction coordinator guaranteeing ACID atomicity with automated backward compensation.

- RBI-mandated Maker-Checker dual control for transfers $\ge$ Rs 50,000.

9. **Stage 9: Unified Results Cockpit & Cryptographic Telemetry:**

- NIST FIPS 202 SHA3-512 Merkle tree ledger providing immutable, tamper-evident audit trails.

---

7. Single Source of Truth (SSOT) Measured Metrics & Verification Reality

All figures below are directly measured from the running codebase and verified by automated audit scripts:

- **Host Specifications:** Windows 11 AMD64, 20 Logical CPU Cores, 15.75 GB RAM, Python 3.14.7.
- **Deterministic Master Verification Tests:** **300 / 300 PASSING** (100.0%) across 3 dedicated suites:

1. `test_kavach_100_suite.py` -> 100 / 100 PASSING

2. `test_banking_100_suite.py` -> 100 / 100 PASSING

3. `test_unified_100_suite.py` -> 100 / 100 PASSING

- **Full Pytest Inventory:** **641 / 641 Total Passing Tests** across 18 test files.
- **Total Registered REST API Endpoints:** **142 Endpoints** (120 Core KAVACH + 15 Bharat Banking `/bank` + 7 Results `/results`).
- **Security Evaluator Performance:**

- Author-Curated V1 Corpus (50 samples): **Precision: 100.0% | Recall: 100.0% | F1: 1.000**

- Held-Out V2 Corpus (45 samples with distractors): **Precision: 89.3% | Recall: 100.0% | F1: 94.3%**

- **Red-Team Cyber Attack Benchmark:** **19 / 19 Vectors Intercepted (100.0% Interception Rate)**.
- **Microsecond Latency Benchmarks (30 runs):**

- PII Detector: **Median 0.02 ms** (p95: 0.02 ms)

- Secret Detector: **Median 0.02 ms** (p95: 0.02 ms)

- Agent Workflow (E2E): **Median 334.41 ms** (p95: 639.22 ms)

---

8. Official University Evaluation Matrix (CSE3101: 30 Marks) & Student Submissions

Section A: Evaluation Criteria Breakdown (Phase-2: 30 Marks Total)

1. **Understanding of Problem Statement & Scope Definition (4 Marks):**

- Full defense: Addresses the 19.7% Slopsquatting vulnerability, DPDP statutory PII compliance, and AST blast-radius containment.

2. **Architecture & Multi-Agent Design Patterns (6 Marks):**

- Full defense: 8-hook Google ADK lifecycle callbacks, CrewAI Stateful Flows DAG (`@start` , `@listen`), FastMCP protocol server, and SAGA distributed transaction coordinator.

3. **Implementation Quality & Functional Completeness (8 Marks):**

- Full defense: 641 passing tests, 142 active REST endpoints, dual-engine privacy router (Ollama air-gapped + Gemini cloud), and Bharat Core Banking system.

4. **Security, Guardrails & Responsible AI (6 Marks):**

- Full defense: Verhoeff Dihedral D_5 Aadhaar validation, Shannon entropy $H > 4.5$ credential redaction, zero-knowledge UUID token vault, and 100% interception of 19 red-team attack vectors.

5. **Observability, Benchmarking & Viva Defense (6 Marks):**

- Full defense: Tamper-evident SHA3-512 Merkle audit ledger, real-time SSE telemetry, live Cyber War Room arena, Forensic DVR Replay, and Inbuilt Codebase IDE modal.

Section B: Student Submissions Checklist

- [x] B.1 Updated Problem Statement (Delivered in documentation and UI)
- [x] B.2 Updated Project Plan & Gantt Timeline (Reflected in Phase 1 -> Phase 2 -> Phase 3 milestones)
- [x] B.3 Work-in-Progress Report & Architecture Diagrams (Delivered in IEEE paper bundle & UI)
- [x] B.4 Code & Demonstration Guide (1-click launchers `run_midsem.bat` and `run_all_services.bat`)
- [x] B.5 Coverage Matrix Linkage (Complete mapping from syllabus topics to code files and lines)

---

9. Faculty Examiner Viva Defense Script (Dr. Shaikh & Mr. Tiwari)

Core Questions & Defenses:

Q1 (Dr. Shaikh): How does KAVACH prevent AI coding agents from introducing malicious dependencies?

> *"Sir, we engineered a dedicated Package Firewall in `package_firewall.py`. When an agent generates code, our AST parser extracts every imported package before writing to disk. We check Python's standard library first (0 ms), our local LRU cache second, and the live PyPI registry third. If PyPI returns an HTTP 404, we immediately terminate execution with exit code 1, preventing Slopsquatting attacks with 100% accuracy."*

Q2 (Mr. Tiwari): How does your Aadhaar detection avoid false alarms on normal 12-digit numbers?

> *"Sir, standard regex like `\d{12}` matches any arbitrary 12 digits, causing high false positive rates. In `detector.py`, we implemented the official UIDAI Verhoeff algorithm using Dihedral Group D_5 permutations. By computing the matrix multiplication $d(i, j)$ across the 12 digits, we mathematically prove whether the number is a valid Aadhaar checksum or simply an invoice number."*

Q3 (Dr. Shaikh): How do CrewAI Stateful Flows differ from standard prompt loops?

> *"Sir, standard multi-agent setups rely on unstructured prompt loops or continuous thread polling which cause race conditions and token waste. In `flows.py`, our CrewAI Flows use an immutable `FlowState` container initialized with `@start` and subscribed tasks with `@listen`. This creates an event-driven Directed Acyclic Graph (DAG) with deterministic state handoffs."*

Q4 (Mr. Tiwari): How do you ensure banking transactions remain atomic during agent execution?

> *"Sir, in `advanced_banking_engines.py`, we implement a distributed SAGA coordinator. If a multi-step transaction fails midway, the coordinator automatically executes backward compensation handlers to rollback previous mutations.

> Furthermore, any transaction exceeding Rs 50,000 is frozen in an escrow quarantine requiring statutory Maker-Checker approval."*

---

10. Operational Guide: How to Run, Test, and Verify Everything

1-Click Bootstrap Launchers:

1. **Launch Mid-Sem Platform (Port 8000):**

```powershell
cd "C:\Users\jaind\OneDrive\Desktop\uni\Ageentic&PRJ-IV\agentic ai mid sem"
.\run_midsem.bat
```

Access in browser: `http://127.0.0.1:8000/`

2. **Launch All Services (Port 8001, 8002, 8003):**

```powershell
cd "C:\Users\jaind\OneDrive\Desktop\uni\Ageentic&PRJ-IV\PRJ-IV &AgenticAi Final Work"
.\run_all_services.bat
```

Access in browser: `http://127.0.0.1:8001/`

3. **Launch Public Cloud Evaluator Tunnel:**

```powershell
cd "C:\Users\jaind\OneDrive\Desktop\uni\Ageentic&PRJ-IV\agentic ai mid sem"
.\run_live_cloud_tunnel.bat
```

4. **Run Master Verification Test Suite (641 Tests):**

```powershell
cd "C:\Users\jaind\OneDrive\Desktop\uni\Ageentic&PRJ-IV\PRJ-IV &AgenticAi Final Work"
python run_all_tests.py
```

---

11. Deliberately Deferred Items, Scope Discipline & Phase 3 Roadmap

To maintain strict scientific integrity and avoid feature bloat during Phase 2 evaluation, the following items are deliberately deferred to Phase 3:

1. **Hardware-Bound HSM Integration:** Software-defined FIPS 140-2 HSM is fully operational; physical PCIe HSM cards deferred to Phase 3 enterprise pilot.

2. **Multi-Region Cross-Cloud Consensus:** Multi-model consensus is fully implemented locally across Gemini, Ollama, and Claude; cross-cloud multi-region active-active clusters deferred to Phase 3.

3. **Automated Dynamic Fuzzing Daemon:** Static AST security fuzzing and 19 red-team vectors are 100% operational; continuous 24/7 autonomous greybox fuzzing daemons deferred to post-capstone deployment.